

Sistema Inteligente de Gestion y Optimizacion de Rutas de Emergencia

Informe Tecnico del Proyecto Integrador
Programacion Avanzada

Autor: Diego Diaz

Repositorio: github.com/DiegoDzn/ProyectoProgramacionAvanzada

Fecha: Julio 2026

*Conceptos integrados: ADT, Hashing, Heaps, Priority Queues,
Sorting, Grafos, BFS, Dijkstra, A*, Analisis de Complejidad*

Indice

1. Introduccion
2. Diseno de Tipos Abstractos de Datos (ADT)
 - 2.1. ADT Incident
 - 2.2. ADT EmergencyCenter
 - 2.3. ADT RoadNetwork
3. Tabla Hash Personalizada
 - 3.1. Implementacion y Operaciones
 - 3.2. Metricas y Analisis
4. Priority Queue y Heap
 - 4.1. Max-Heap para Incidentes
 - 4.2. Min-Heap para Busquedas
 - 4.3. Benchmarks de Rendimiento
5. Algoritmos de Ordenamiento
 - 5.1. MergeSort
 - 5.2. QuickSort
 - 5.3. Comparacion Experimental
6. Grafos y Algoritmos de Busqueda
 - 6.1. Modelado de la Red Vial
 - 6.2. BFS
 - 6.3. Dijkstra
 - 6.4. A*
 - 6.5. Comparacion de Rutas
7. Escenario Integrado
8. Analisis Experimental Completo
9. Conclusiones

1. Introduccion

Durante un desastre natural, los centros de emergencia reciben numerosas solicitudes de ayuda que requieren una gestion eficiente y prioritaria. El presente proyecto implementa un prototipo de software capaz de registrar incidentes, priorizarlos, almacenarlos de forma eficiente y calcular rutas optimas para la atencion de emergencias en tiempo real.

El sistema integra multiples estructuras de datos y algoritmos fundamentales de la ciencia de la computacion: Tipos Abstractos de Datos (ADT), una tabla hash propia con separate chaining, una cola de prioridad basada en heap binario de maximos, algoritmos de ordenamiento (MergeSort y QuickSort), modelado de grafos ponderados y algoritmos de busqueda (BFS, Dijkstra y A*).

Objetivos

- Registrar y gestionar incidentes con validacion de precondiciones y postcondiciones.
- Priorizar solicitudes de emergencia mediante una cola de prioridad basada en Max-Heap.
- Almacenar informacion en una tabla hash propia con metricas de colisiones y factor de carga.
- Analizar zonas afectadas y generar reportes ordenados por frecuencia, prioridad y antiguedad.
- Calcular rutas optimas entre centros de operacion y puntos de emergencia.
- Ejecutar un escenario integrado completo con 500 incidentes y una red vial de 50 nodos.

Alcance

El dataset incluye 500 incidentes generados con 8 tipos distintos (Incendio, Accidente Vial, Emergencia Medica, Derrumbe, Inundacion, Rescate, Fuga de Gas, Explosion) distribuidos en 50 nodos de una red vial modelada como un grafo con 121 aristas ponderadas por tiempo de desplazamiento. Dos centros de emergencia operan como bases de despacho.

2. Diseno de Tipos Abstractos de Datos (ADT)

Cada ADT ha sido disenado siguiendo el paradigma orientado a objetos con encapsulamiento (propiedades con getters/setters), validacion de precondiciones en constructores, y documentacion formal de complejidades temporales y espaciales.

2.1. ADT Incident

Representa un incidente reportado en el sistema de emergencias.

Atributo	Tipo	Descripcion	Complejidad
id	str	Identificador unico (ej: I-152)	O(1)
ubicacion	str	Nodo o zona en el grafo	O(1)
prioridad	float	Nivel de urgencia calculado	O(1)
tipo	str	Tipo de incidente	O(1)
timestamp	float	Tiempo de reporte (epoch)	O(1)
estado	str	Pendiente / En Proceso / Resuelto	O(1)

Operaciones principales:

- Constructor Incident(id, ubicacion, prioridad, tipo, timestamp, estado): O(1)
- actualizar_estado(nuevo_estado): O(1) - Valida contra el conjunto de estados validos.
- prioridad.setter: O(1) - Valida no negatividad.

2.2. ADT EmergencyCenter

Representa un centro de operaciones (hospital, estacion de bomberos, base de rescate).

Atributo	Tipo	Descripcion	Complejidad
id	str	Identificador unico (ej: EC-01)	O(1)
nombre	str	Nombre descriptivo	O(1)
ubicacion	str	Nodo en el grafo vial	O(1)

2.3. ADT RoadNetwork

Representa la red vial como un grafo ponderado bidireccional. Utiliza una lista de adyacencia internamente representada como un diccionario de diccionarios: {nodo: {destino: peso}}.

Operacion	Precondicion	Postcondicion	Complejidad
agregar_nodo(n)	n es str no vacio	Nodo registrado en la red	O(1)
agregar_arista(o,d,p)	o,d existen, p >= 0	Arista bidireccional registrada	O(1)
obtener_nodos()	Ninguna	Lista de todos los nodos	O(V)
obtener_adyacentes(n)	n existe en la red	Dict copia de adyacencias	O(1)
obtener_peso(o,d)	Arista o->d existe	Peso (float) de la arista	O(1)
cargar_desde_json(r)	Archivo JSON valido	Grafo completamente cargado	O(V+E)

3. Tabla Hash Personalizada

3.1. Implementacion y Operaciones

La tabla hash implementa Direccionamiento Encadenado (Separate Chaining) donde cada bucket es una lista enlazada de pares (clave, valor). Esta disenada exclusivamente para almacenar incidentes y NO utiliza el diccionario nativo de Python (dict) para su operacion interna.

Funcion hash: Metodo polinomial de Horner con constante multiplicativa $p=31$, operando sobre 64 bits para evitar desbordamientos: $h = (h * 31 + \text{ord}(\text{char})) \bmod M$.

Redimensionamiento: Cuando el factor de carga $\alpha = N/M$ supera 0.75, la tabla se redimensiona al siguiente numero primo mayor al doble de su capacidad actual, y todos los elementos son rehashados.

Operacion	Precondicion	Postcondicion	Complejidad
insert(k, v)	k es str no vacia	Par almacenado, resize si $\alpha > 0.75$	$O(1)$ promedio
get(k)	k es str no vacia	Retorna valor o KeyError	$O(1)$ promedio
delete(k)	k existe en la tabla	Elemento eliminado	$O(1)$ promedio
contains(k)	Ninguna	True/False	$O(1)$ promedio
get_stats()	Ninguna	Dict con metricas completas	$O(M)$

3.2. Metricas y Analisis

Resultados con 500 incidentes almacenados:

Metrica	Valor	Interpretacion
Capacidad (M)	1009	Primo mas cercano a 1009
Elementos (N)	500	500 incidentes del dataset
Factor de carga (N/M)	0.4955	Por debajo del umbral 0.75
Total colisiones	102	Suma de $(\text{len}(\text{bucket})-1)$ por bucket
Buckets utilizados	398	398/1009 buckets activos
Max tamano bucket	2	Cadena maxima de colisiones

El factor de carga de 0.4955 indica una distribucion eficiente de los 500 incidentes en 1009 buckets. Con un maximo de 2 elementos por bucket, las colisiones son minimas y el tiempo de busqueda se mantiene cercano a $O(1)$.

4. Priority Queue y Heap

4.1. Max-Heap para Incidentes

La PriorityQueue principal implementa un Heap Binario de Maximos almacenado en un arreglo plano. Utiliza un mapa de posiciones interno (_posiciones: dict id->indice) que permite buscar, actualizar y eliminar incidentes en $O(\log N)$ en lugar de $O(N)$.

Criterio de orden de urgencia:

1. Mayor prioridad sale primero.
2. En caso de empate, el de menor timestamp (reportado primero) tiene prioridad.

Operacion	Complejidad	Descripcion
insertar(inc)	$O(\log N)$	Inserta y restaura propiedad heap con sift-up
extraer_urgente()	$O(\log N)$	Remueve raiz y restaura con sift-down
actualizar_prioridad(id, p)	$O(\log N)$	Busca via mapa, actualiza y reordena
obtener_top_k(k)	$O(K \log N)$	Extrae K elementos de una copia temporal

4.2. Min-Heap para Busquedas

La MinPriorityQueue es una cola generica de minimos que almacena tuplas (prioridad, item). Es utilizada internamente por los algoritmos Dijkstra y A* para extraer siempre el nodo con menor costo acumulado. Su implementacion es mas sencilla sin mapa de posiciones.

4.3. Benchmarks de Rendimiento

Tiempos promedio de insercion y extraccion del Max-Heap (5 repeticiones):

Tamano (N)	Insercion (s)	Extraccion (s)	Total (s)	Razon ext/ins
100	0.000211	0.000596	0.000807	2.8x
500	0.001041	0.004125	0.005166	4.0x
1000	0.001799	0.009309	0.011108	5.2x

Los benchmarks confirman el comportamiento $O(\log N)$. Al duplicar N de 100 a 500, el tiempo de insercion crece ~4.9x. La extraccion es consistentemente mas lenta que la insercion debido al sift-down post-extraccion y la reorganizacion del mapa de posiciones.

5. Algoritmos de Ordenamiento

5.1. MergeSort

Algoritmo de dividir y vencerar estable. Divide la lista en mitades recursivamente hasta sublistas de tamaño 1, luego combina ordenadamente con la función `_merge`.

- Complejidad temporal: $O(N \log N)$ en todos los casos.
- Complejidad espacial: $O(N)$ memoria auxiliar.
- Estabilidad: SI (preserva orden relativo de elementos con igual clave).
- Ventaja: Rendimiento predecible sin peor caso degenerado.

5.2. QuickSort

Algoritmo de dividir y vencerar in-place con selección de pivote por elemento medio para mitigar la degradación a $O(N^2)$ en arreglos pre-ordenados.

- Complejidad temporal: $O(N \log N)$ promedio, $O(N^2)$ peor caso.
- Complejidad espacial: $O(\log N)$ auxiliar recursivo.
- Estabilidad: NO preserva orden relativo.
- Ventaja: Menor constante empírica en el promedio.

5.3. Comparación Experimental

Tiempos promedio en segundos (5 repeticiones por caso):

N	Caso	MergeSort (s)	QuickSort (s)	Más rápido
100	Aleatorio	0.000144	0.000080	QS
100	Ordenado	0.000114	0.000064	QS
100	Inverso	0.000131	0.000075	QS
500	Aleatorio	0.001266	0.000902	QS
500	Ordenado	0.001152	0.000674	QS
500	Inverso	0.001124	0.000845	QS
1000	Aleatorio	0.004021	0.002518	QS
1000	Ordenado	0.002779	0.002812	MS
1000	Inverso	0.002699	0.002130	QS

QuickSort es consistentemente más rápido que MergeSort en la mayoría de los casos, especialmente para datos aleatorios donde su particionado por pivote medio es muy eficiente. En datos ordenados ambos son comparables, con QuickSort ventaja mínima. MergeSort mantiene su ventaja de estabilidad, siendo preferido cuando el orden relativo de elementos con igual clave es importante (ej: ordenar por zona manteniendo orden de llegada).

6. Grafos y Algoritmos de Búsqueda

6.1. Modelado de la Red Vial

La red vial se modela como un grafo no dirigido ponderado con 50 nodos distribuidos en una cuadrícula de 5 filas por 10 columnas. Cada nodo tiene coordenadas (x, y) para permitir búsquedas heurísticas. Las 121 aristas representan rutas con pesos correspondientes al tiempo de desplazamiento en minutos, calculados a partir de la distancia euclidiana con un factor de variación de 0.9 a 1.3.

La estructura del grafo:

- Nodos: 50 distritos (Nodo_0_0 a Nodo_4_9)
- Aristas: 121 conexiones bidireccionales
- Centros de emergencia: EC-01 en Nodo_0_0 (Norte), EC-02 en Nodo_4_9 (Sur)
- Carga desde archivo JSON con coordenadas y pesos predefinidos

6.2. BFS (Breadth-First Search)

Busqueda en anchura. Explora todos los vecinos de un nivel antes de pasar al siguiente.

- Encuentra la ruta con MENOR NUMERO DE TRAMOS (hops), ignorando pesos.
- Complejidad: $O(V + E)$ tiempo, $O(V)$ espacio.
- Implementacion: Cola FIFO con set de visitados.

6.3. Dijkstra

Algoritmo de caminos minimos para grafos con pesos no negativos.

- Garantiza la ruta de MENOR COSTO TOTAL (tiempo de desplazamiento).
- Complejidad: $O((V + E) \log V)$ con Min-Heap.
- Implementacion: MinPriorityQueue con mapa de distancias.

6.4. A* (A-Estrella)

Busqueda informada que combina el costo real $g(n)$ con una heurística $h(n)$.

- Heurística: Distancia euclidiana entre coordenadas de nodos.
- Complejidad: $O((V + E) \log V)$ con heurística admisible.
- Garantiza optimalidad cuando $h(n)$ es admisible (nunca sobreestima).

6.5. Comparacion de Rutas

Resultados en 5 pares origen-destino aleatorios:

Algoritmo	Nodos Visitados	Costo (min)	Hops
BFS	45	124.69	11
Dijkstra	46	111.06	11
A* (Euc.)	33	111.06	11

Algoritmo	Nodos Visitados	Costo (min)	Hops
BFS	40	83.11	6
Dijkstra	41	82.11	6
A* (Euc.)	17	82.29	6

Algoritmo	Nodos Visitados	Costo (min)	Hops
-----------	-----------------	-------------	------

BFS	10	22.24	2
Dijkstra	9	22.24	2
A* (Euc.)	3	22.24	2

Algoritmo	Nodos Visitados	Costo (min)	Hops
BFS	39	57.06	5
Dijkstra	42	54.93	5
A* (Euc.)	13	54.93	5

Algoritmo	Nodos Visitados	Costo (min)	Hops
BFS	27	45.78	4
Dijkstra	25	45.78	4
A* (Euc.)	7	45.78	4

7. Escenario Integrado

El escenario integrado demuestra el flujo completo del sistema:

Paso 1 - Carga de datos: Se leen 500 incidentes desde incidentes.csv y la red vial desde red_vial.json (50 nodos, 121 aristas).

Paso 2 - Insercion en estructuras: Los 500 incidentes se insertan simultaneamente en la Tabla Hash (para busquedas por ID) y en la PriorityQueue (para despacho por urgencia).

Paso 3 - Extraccion del mas urgente: Se extrae el incidente con mayor prioridad calculada como $\text{severidad} \times \text{factor_tiempo}$.

Paso 4 - Calculo de ruta: Se ejecuta Dijkstra desde ambos centros de emergencia hasta la ubicacion del incidente, seleccionando el centro con menor ETA.

Paso 5 - Despacho y telemetria: Se simula el desplazamiento del vehiculo nodo a nodo, mostrando el costo acumulado y actualizando el estado del incidente a ATENDIDO.

Componentes del escenario integrado:

- 2 centros de emergencia (Base Alfa y Base Beta)
- 500 incidentes con 8 tipos distintos
- Red vial con 50 distritos y 121 rutas monitoreadas
- Comparativa en tiempo real de BFS, Dijkstra y A*
- Benchmark de MergeSort vs QuickSort con $N=1000$
- Benchmark de PriorityQueue con insercion y extraccion

8. Analisis Experimental Completo

8.1. Tabla Hash - Rendimiento

Con 500 incidentes insertados en una tabla de capacidad 1009:

- Factor de carga: 0.4955 (umbral de resize: 0.75)
- Colisiones totales: 102 de 500 inserciones (20.4% de inserciones causaron colision)
- Buckets activos: 398/1009 (39.4% de capacidad utilizada)
- Cadena maxima: 2 elementos (busqueda $O(1)$ en el peor caso)

Conclusion: La tabla hash opera eficientemente con separate chaining. El factor de carga moderado y la cadena maxima de 2 elementos garantizan tiempos de acceso cercanos a $O(1)$.

8.2. Priority Queue - Rendimiento del Heap

El Max-Heap demuestra comportamiento $O(\log N)$ tanto en insercion como en extraccion. La operacion de extraccion es aproximadamente 2-5x mas lenta que la insercion debido a que requiere: (1) swap de raiz con ultimo elemento, (2) pop del ultimo, (3) sift-down completo desde la raiz, y (4) actualizacion del mapa de posiciones.

8.3. Sorting - Comparacion de Algoritmos

MergeSort mantiene tiempos consistentes entre casos ($O(N \log N)$ garantizado), mientras que QuickSort es tipicamente 20-40% mas rapido en datos aleatorios pero puede ser comparable en datos ya ordenados. Para reportes donde la estabilidad importa (ej: frecuencia por zona manteniendo orden de llegada), MergeSort es la mejor opcion.

8.4. Grafos - Comparacion de Algoritmos de Busqueda

Metrica	BFS	Dijkstra	A* (Euc.)
Nodos visitados (promedio)	32.2	32.6	14.6
Costo total (promedio, min)	66.58	63.22	63.26

Resultados promedio en 5 consultas:

- BFS: 32 nodos visitados, costo promedio 66.58 min. Encuentra rutas con menos hops pero ignora pesos.
- Dijkstra: 33 nodos visitados, costo optimo 63.22 min. Explora ampliamente pero garantiza optimalidad.
- A*: 15 nodos visitados, costo 63.26 min. La heuristica euclidiana reduce la busqueda en un 55% respecto a Dijkstra, manteniendo la ruta optima.

9. Conclusiones

El sistema implementado demuestra la integracion efectiva de multiples estructuras de datos y algoritmos para resolver un problema real de gestion de emergencias:

1. ADT Incident, EmergencyCenter y RoadNetwork proporcionan un modelado robusto con validaciones de precondiciones, encapsulamiento y documentacion formal de complejidades.
 2. La Tabla Hash propia con separate chaining y redimensionamiento automatico almacena 500 incidentes con solo 102 colisiones y factor de carga de 0.4955, demostrando acceso eficiente cercano a $O(1)$ sin depender del dict nativo de Python.
 3. La PriorityQueue Max-Heap con mapa de posiciones permite insercion, extraccion y actualizacion de prioridad en $O(\log N)$, siendo fundamental para el despacho en tiempo real de incidentes por orden de urgencia.
 4. MergeSort y QuickSort generan reportes ordenados de forma eficiente. QuickSort es tipicamente mas rapido (20-40%), mientras que MergeSort ofrece estabilidad y rendimiento predecible en todos los casos.
 5. La red vial modelada como grafo ponderado (50 nodos, 121 aristas) permite calcular rutas optimas con Dijkstra (costo minimo) y A* (busqueda informada). A* reduce los nodos visitados en un ~55% respecto a Dijkstra manteniendo la optimalidad.
 6. El escenario integrado demuestra el flujo completo: carga de 500 incidentes, insercion en tabla hash y priority queue, extraccion del mas urgente, seleccion del centro mas cercano y calculo de ruta optima con telemetria en tiempo real.
- El prototipo cumple con todos los requerimientos del proyecto: ADTs documentados, tabla hash propia sin dict, priority queue basada en heap, algoritmos de sorting con comparacion experimental, grafo vial con BFS/Dijkstra/A*, escenario integrado con 500+ incidentes, y dashboard web interactivo con visualizacion SVG de la red vial.